

Proving the correctness of a compiler

A Lean 4 development

Course material after Xavier Leroy, EUTypes 2019

Lean 4 port and slides

Slides written for the Lean 4 port in this repository.

Original Coq course by Xavier Leroy, Collège de France and Inria Paris.

Why prove a compiler correct?

A compiler sits between the program you reasoned about and the program that actually runs.

- You verify the source. The machine runs the target.
- Every guarantee you established — by testing, by static analysis, by proof — is only as good as the translation.
- Compilers are not bug-free. Random-testing campaigns find miscompilations in every mainstream compiler.

A miscompilation is especially nasty: the source is right, the reasoning is right, and the bug is in the tool you trusted to be invisible.

What would a theorem look like?

Let S be a source program and $C = \text{compile}(S)$ its compiled form.

Semantic preservation, informally:

every observable behaviour of C is an observable behaviour of S .

For our language the observable behaviours are:

- *terminating* with a given final store,
- *diverging* (running forever),
- *going wrong* (getting stuck).

We will prove: if S terminates in store s' then C halts in s' ; and if S diverges then C diverges.
Compiled code never goes wrong.

Two ingredients

To even *state* such a theorem we need formal semantics for both languages. To *prove* it we need a proof assistant, because the case analyses are large and unforgiving.

This development uses **Lean 4**, with no library dependencies beyond the core.

Everything is machine-checked: no sorry, and the headline theorems depend only on `propext`, `Classical.choice` and `Quot.sound`.

Roadmap

1. The source language: **IMP**
2. The target language: a **stack machine**
3. The **compilation scheme**
4. First correctness proofs — terminating programs
6. Small-step semantics for IMP
7. Full correctness — terminating *and* diverging programs
8. An optimisation: **constant propagation**
9. More about **fixpoints**
10. **Liveness analysis** and dead code elimination

(Section numbering follows the original course.)

1.1 Arithmetic expressions

```
inductive aexp where
  | const (n : Int)
  | var    (x : Ident)
  | plus   (1a 2a : aexp)
  | minus  (1a 2a : aexp)
```

A *store* maps variables to their current values, and evaluation is an ordinary recursive function — a denotational semantics:

```
def Store : Type := Ident → Int

def aeval (s : Store) : aexp → Int
  | .const n    => n
  | .var x      => s x
  | .plus 1a 2a => aeval s 1a + aeval s 2a
  | .minus 1a 2a => aeval s 1a - aeval s 2a
```

What an evaluation function is good for

Run a given expression in a given store:

```
example : aeval (fun _ => 2)
  (.plus (.var "x") (.minus (.var "x") (.const 1))) = 3 := by native_decide
```

Prove a property of a given expression, for all stores:

```
theorem aeval_xplus1 (s : Store) (x : Ident) :
  aeval s (.plus (.var x) (.const 1)) > aeval s (.var x)
```

Prove a meta-property, for all expressions at once — the value of an expression depends only on its free variables:

```
theorem aeval_free {s1 s2 : Store} :  $\forall a : \text{aexp},$ 
   $\forall (x, \text{freeInAexp } x \ a \rightarrow s1 \ x = s2 \ x) \rightarrow \text{aeval } s1 \ a = \text{aeval } s2 \ a$ 
```

This last kind is what compiler verification is made of.

1.3 Boolean expressions

```
inductive bexp where  
  | true | false  
  | equal      (₁a ₂a : aexp)  
  | lessequal (₁a ₂a : aexp)  
  | not (₁b : bexp)  
  | and (₁b ₂b : bexp)
```

Everything else is a *derived form* — no new syntax, no new semantics to prove sound:

```
def bexp.greater (₁a ₂a : aexp) : bexp := .not (.lessequal ₁a ₂a)  
def bexp.or (₁b ₂b : bexp) : bexp := .not (.and (.not ₁b) (.not ₂b))
```

```
theorem beval_or (s : Store) (₁b ₂b : bexp) :  
  beval s (bexp.or ₁b ₂b) = (beval s ₁b || beval s ₂b)
```

1.4 Commands

inductive com **where**

```
| skip
| assign      (x : Ident) (a : aexp)
| seq         (c1 c2 : com)
| ifthenelse  (b : bexp) (c1 c2 : com)
| while       (b : bexp) (c : com)
```

infixr:80 " ;; " => com.seq

Euclidean division by repeated subtraction:

def euclideanDivision : com :=

```
.assign "r" (.var "a") ;;
.assign "q" (.const 0) ;;
.while (.lessequal (.var "b") (.var "r"))
  (.assign "r" (.minus (.var "r") (.var "b"))) ;;
  .assign "q" (.plus (.var "q") (.const 1)))
```

The naive idea, and why it fails

Why not an evaluation function for commands too?

```
def cexec (s : Store) : com → Store
| .skip      => s
| .assign x a => update x (aeval s a) s
| .seq 1c 2c  => cexec (cexec s 1c) 2c
| .while b 1c => if beval s b then cexec (cexec s 1c) (.while b 1c) else s
```

The naive idea, and why it fails

Why not an evaluation function for commands too?

```
def cexec (s : Store) : com → Store
| .skip      => s
| .assign x a => update x (aeval s a) s
| .seq 1c 2c  => cexec (cexec s 1c) 2c
| .while b 1c => if beval s b then cexec (cexec s 1c) (.while b 1c) else s
```

Rejected — rightly. Every Lean function is total, but while true do skip does not terminate.

Worse: IMP is Turing-complete (unbounded loops, unbounded integers), so *no* computable function can return the final store when one exists and report failure otherwise.

Big-step (natural) semantics

Replace the function by a *relation*: $\text{cexec } s \ c \ s'$ holds exactly when running c from s terminates in s' .

```
inductive cexec : Store → com → Store → Prop where
  | skip    (s) : cexec s .skip s
  | assign (s x a) : cexec s (.assign x a) (update x (aeval s a) s)
  | seq : cexec s 1c s' → cexec s' 2c s'' → cexec s (1c ;; 2c) s''
  | ifthenelse : cexec s (if beval s b then 1c else 2c) s' →
    cexec s (.ifthenelse b 1c 2c) s'
  | while_done : beval s b = false → cexec s (.while b c) s
  | while_loop : beval s b = true → cexec s c s' →
    cexec s' (.while b c) s'' → cexec s (.while b c) s''
```

A derivation is a *finite* tree. Finiteness is exactly what rules out non-terminating executions.

Divergence has no derivation

theorem cexec_infinite_loop (s : Store) :
 $\neg \exists s', \text{cexec } s \text{ (.while .true .skip)} s'$

Proof: strengthen to “no derivation concludes about while true do skip”, then induct. The while_done case contradicts beval s .true = false; the while_loop case appeals to the induction hypothesis.

Divergence has no derivation

theorem `cexec_infinite_loop` (`s` : Store) :
 $\neg \exists s', \text{cexec } s \text{ (.while .true .skip)} s'$

Proof: strengthen to “no derivation concludes about while true do skip”, then induct. The while_done case contradicts `beval s .true = false`; the while_loop case appeals to the induction hypothesis.

This is the strength *and* the weakness of big-step semantics.

- Strength: termination is characterised precisely.
- Weakness: divergence and going-wrong are both “no derivation”. The semantics cannot tell them apart.

We will need small-step semantics in §6 for exactly this reason.

An interpreter, for testing

The naive idea is salvageable if we bound the recursion depth:

```
def cexecBounded : Nat → Store → com → Option Store
| 0, _, _ => none
| fuel + 1, s, c => match c with
| .skip => some s
| .seq ₁c ₂c => match cexecBounded fuel s ₁c with
| none => none | some s' => cexecBounded fuel s' ₂c
| ...
```

Useless as a semantics, invaluable for trying programs out:

```
example : (let s := update "a" 14 (update "b" 3 (fun _ => 0))
  (cexecBounded 100 s euclideanDivision).map (fun s' => (s' "q", s' "r")))
= some (4, 2) := by native_decide
```

2.1 A stack machine

Like an old RPN pocket calculator: a stack holds intermediate results; each instruction pops its arguments and pushes its result.

inductive instr **where**

```
| const (n : Int)      -- push n
| var    (x : Ident)   -- push the value of x
| setvar (x : Ident)   -- pop, assign to x
| add     -- pop two, push their sum
| opp     -- pop one, push its opposite
| branch (d : Int)     -- skip d instructions
| beq    (i d o d : Int) -- pop two; skip i d if equal, o d if not
| ble    (i d o d : Int) -- pop two; skip i d if ≤, o d if >
| halt   -- stop
```

abbrev Code := List instr

Note: no subtraction instruction. We will compile $a - b$ as $a + (-b)$.

2.2 Operational semantics

A configuration is a program counter, a stack, and a store. The code is fixed throughout an execution.

abbrev Config := Int × Stack × Store

```
def instrAt : Code → Int → Option instr
  | [], _      => none
  | i :: C', pc => if pc = 0 then some i else instrAt C' (pc - 1)
```

Program counters are *integers*, since branches may go backwards.

```
inductive transition (C : Code) : Config → Config → Prop where
  | const : instrAt C pc = some (.const n) →
    transition C (pc, stk, s) (pc + 1, n :: stk, s)
  | branch : instrAt C pc = some (.branch d) → pc' = pc + 1 + d →
    transition C (pc, stk, s) (pc', stk, s)
  | ...
```

Three machine behaviours

halt has *no* transition — that is what makes it stop the machine rather than get it stuck.

```
def machineTerminates (C) (sInit sFinal : Store) : Prop :=  
  ∃ pc, transitions C (0, [], sInit) (pc, [], sFinal)  
    ∧ instrAt C pc = some .halt
```

```
def machineDiverges (C) (sInit : Store) : Prop :=  
  Infseq (transition C) (0, [], sInit)
```

```
def machineGoesWrong (C) (sInit : Store) : Prop :=  
  ∃ pc stk s, transitions C (0, [], sInit) (pc, stk, s)  
    ∧ Irred (transition C) (pc, stk, s)  
    ∧ (instrAt C pc ≠ some .halt ∨ stk ≠ [])
```

Execution starts at $pc = 0$ with an empty stack and succeeds when it reaches a halt with an empty stack.

3.1 Expressions: reverse Polish notation

```
def compileAexp : aexp → Code
  | .const n    => [.const n]
  | .var x      => [.var x]
  | .plus  1a 2a => compileAexp 1a ++ compileAexp 2a ++ [.add]
  | .minus 1a 2a => compileAexp 1a ++ compileAexp 2a ++ [.opp, .add]
```

Informal specification. The code for a

- executes in sequence — no branches;
- leaves the value of a on top of the stack;
- leaves the store unchanged.

This is the contract we will turn into a theorem in §4.

3.2 Boolean expressions: branch, don't compute

We could push 0 or 1 and test it. It is shorter and faster to generate code that *branches directly*, never materialising the Boolean.

`compileBexp b d1 d0` produces code that

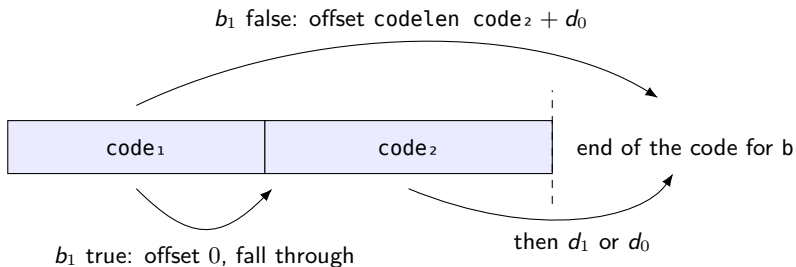
- skips `d1` instructions forward if `b` is true,
- skips `d0` instructions forward if `b` is false,
- leaves the stack and the store unchanged.

```
def compileBexp : bexp → Int → Int → Code
| .true, 1d, _ => if 1d = 0 then [] else [.branch 1d]
| .false, _, 0d => if 0d = 0 then [] else [.branch 0d]
| .equal 1a 2a, 1d, 0d =>
  compileAexp 1a ++ compileAexp 2a ++ [.beq 1d 0d]
| .not 1b, 1d, 0d => compileBexp 1b 0d 1d
| ...
```

Negation costs nothing at all: just swap the two offsets.

The offsets for conjunction

```
| .and 1b 2b, 1d, 0d =>  
  let 2code := compileBexp 2b 1d 0d  
  let 1code := compileBexp 1b 0 (codelen 2code + 0d)  
  1code ++ 2code
```



If b_1 is false the whole conjunction is false, so we must jump over `code2` *and then* take b 's own false branch: hence $\text{codelen code}_2 + d_0$.

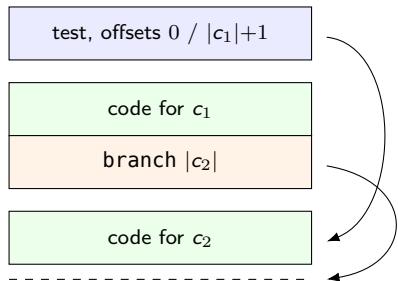
3.3 Commands

Specification. The code for c updates the store as c prescribes, preserves the stack, and falls through to the instruction following the generated code.

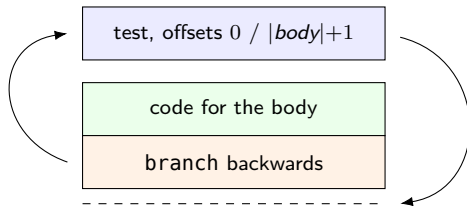
```
def compileCom : com → Code
| .skip => []
| .assign x a => compileAexp a ++ [.setvar x]
| .seq 1c 2c => compileCom 1c ++ compileCom 2c
| .ifthenelse b ifso ifnot =>
    compileBexp b 0 (codelen (compileCom ifso) + 1)
    ++ compileCom ifso
    ++ instr.branch (codelen (compileCom ifnot)) :: compileCom ifnot
| .while b body =>
    let codeBody := compileCom body
    let codeTest := compileBexp b 0 (codelen codeBody + 1)
    codeTest ++ codeBody
    ++ [.branch (-(codelen codeTest + codelen codeBody + 1))]
```

The two control-flow layouts

if-then-else



while



The whole program appends a `halt` so that it stops cleanly: `compileProgram p = compileCom p ++ [halt]`.

Examples

```
compileProgram (.assign "x" (.plus (.var "x") (.const 1)))  
  = [.var "x", .const 1, .add, .setvar "x", .halt]
```

```
compileProgram (.while .true .skip)  
  = [.branch (-1), .halt]
```

```
compileProgram (.ifthenelse (.equal (.var "x") (.const 1))  
                             (.assign "x" (.const 0)) .skip)  
  = [.var "x", .const 1, .beq 0 3,  
     .const 0, .setvar "x", .branch 0, .halt]
```

The last one shows a small inefficiency: branch 0 does nothing. A smart constructor fixes it — see the exercises.

```
def smartBranch (d : Int) : Code := if d = 0 then [] else [.branch d]
```

Reasoning about code fragments

The code for a sub-expression sits *inside* the code for the whole program, at some offset.

```
def CodeAt (C : Code) (pc : Int) (C' : Code) : Prop :=  
   $\exists \text{ } _1C \text{ } _3C, C = \text{ } _1C ++ C' ++ \text{ } _3C \wedge pc = \text{codeLen } _1C$ 
```

Navigation lemmas let us descend into a fragment:

```
theorem CodeAt.head : CodeAt C pc (i :: C')  $\rightarrow$  instrAt C pc = some i  
theorem CodeAt.app_left  : CodeAt C pc ( $_1D ++ \text{ } _2D$ )  $\rightarrow$  CodeAt C pc  $_1D$   
theorem CodeAt.app_right : CodeAt C pc ( $_1D ++ \text{ } _2D$ )  $\rightarrow$   
  CodeAt C (pc + codeLen  $_1D$ )  $_2D$ 
```

Every correctness statement below is relative to such a hypothesis.

4.1 Expressions

The informal contract, now a theorem:

```
theorem compileAexp_correct {C : Code} {s : Store} (a : aexp) :  
  ∀ {pc pc' : Int} {stk : Stack},  
  CodeAt C pc (compileAexp a) →  
  pc' = pc + codelen (compileAexp a) →  
  transitions C (pc, stk, s) (pc', aeval s a :: stk, s)
```

Read it as: starting at the beginning of the generated code with any stack, the machine runs to the end of that code, having pushed `aeval s a` and changed nothing else.

Proof: induction on `a`. For `plus`, chain the run of `compileAexp a1`, then that of `compileAexp a2`, then one `add`.

The target program counter is a *variable* `pc'` constrained by an equation. That is what lets the pieces compose without rewriting arithmetic at every step.

4.1 Boolean expressions

theorem compileBexp_correct {C : Code} {s : Store} (b : bexp) :
 $\forall (i d \text{ } o d : \text{Int}) \{pc \text{ } pc' : \text{Int}\} \{stk : \text{Stack}\},$
 CodeAt C pc (compileBexp b i d o d) $\rightarrow$
 pc' = pc + codelen (compileBexp b i d o d)
 + (if beval s b then i d else o d) $\rightarrow$
 transitions C (pc, stk, s) (pc', stk, s)

The statement is the informal specification, verbatim.

The and case is where the offsets earn their keep: after running `code1`, case on `beval s b1`. If true we are exactly at the start of `code2` and apply the induction hypothesis; if false we have already landed at the right place and take zero further steps.

4.2 Commands, terminating case

theorem compileCom_correct_terminating {C : Code} {s c s'}
 (h : cexec s c s') :
 ∀ {pc pc' : Int} {stk : Stack},
 CodeAt C pc (compileCom c) → pc' = pc + codelen (compileCom c) →
 transitions C (pc, stk, s) (pc', stk, s')

Induction on the *big-step derivation*, not on the command — the `while_loop` case needs the induction hypothesis for the loop itself, re-entered at the same pc.

theorem compileProgram_correct_terminating {s c s'} (h : cexec s c s') :
 machineTerminates (compileProgram c) s s'

What we have, and what is missing

Have. If the source terminates in s' , the compiled program halts in s' .

Missing. Everything about programs that do *not* terminate.

The big-step semantics simply has nothing to say: `while true do skip` has no derivation, and so the theorem above is vacuous for it. A compiler that turned every loop into `halt` would satisfy it.

What we have, and what is missing

Have. If the source terminates in s' , the compiled program halts in s' .

Missing. Everything about programs that do *not* terminate.

The big-step semantics simply has nothing to say: `while true do skip` has no derivation, and so the theorem above is vacuous for it. A compiler that turned every loop into `halt` would satisfy it.

To rule that out we need a semantics in which divergence is a positive statement rather than an absence. That means small-step.

6.1 Reduction semantics

$\text{red } (c, s) (c', s')$: one elementary step of c in store s , leaving residual command c' .

inductive $\text{red} : \text{com} \times \text{Store} \rightarrow \text{com} \times \text{Store} \rightarrow \text{Prop}$ **where**

- | $\text{assign } (x \ a \ s) : \text{red } (. \text{assign } x \ a, s) (. \text{skip}, \text{update } x \ (\text{aeval } s \ a) \ s)$
- | $\text{seq_done } (c \ s) : \text{red } (. \text{skip} \ ;\ ; \ c, s) (c, s)$
- | $\text{seq_step} : \text{red } ({}_1c, {}_1s) ({}_2c, {}_2s) \rightarrow \text{red } ({}_1c \ ;\ ; \ c, {}_1s) ({}_2c \ ;\ ; \ c, {}_2s)$
- | $\text{ifthenelse } (b \ {}_1c \ {}_2c \ s) :$
 - $\text{red } (. \text{ifthenelse } b \ {}_1c \ {}_2c, s) ((\text{if } \text{beval } s \ b \ \text{then } {}_1c \ \text{else } {}_2c), s)$
- | $\text{while_done} : \text{beval } s \ b = \text{false} \rightarrow \text{red } (. \text{while } b \ c, s) (. \text{skip}, s)$
- | $\text{while_loop} : \text{beval } s \ b = \text{true} \rightarrow \text{red } (. \text{while } b \ c, s) (c \ ;\ ; \ . \text{while } b \ c, s)$

Now a diverging program is one with an infinite sequence of steps — a positive statement.

IMP programs cannot go wrong

theorem red_progress (c : com) (s : Store) :
c = .skip $\vee \exists c' s', \text{red } (c, s) (c', s')$

def goesWrong (c : com) (s : Store) : Prop :=
 $\exists c' s', \text{Star red } (c, s) (c', s') \wedge \text{Irred red } (c', s') \wedge c' \neq \text{.skip}$

theorem not_goesWrong (c : com) (s : Store) : $\neg \text{goesWrong } c \ s$

Every non-skip command can take a step, so the only irreducible state is skip. Combined with compiler correctness, this is what tells us compiled code never gets stuck either.

Big-step and small-step agree

theorem cexec_to_reds {s c s'} (h : cexec s c s') :
Star red (c, s) (.skip, s')

theorem reds_to_cexec {s c s'} (h : Star red (c, s) (.skip, s')) :
cexec s c s'

The first is a routine induction on the derivation. The second rests on a key lemma — one reduction step followed by a big-step execution collapses into a single big-step execution:

theorem red_append_cexec (h : red p q) :
 $\forall \{s'\}, \text{cexec } q.2 \ q.1 \ s' \rightarrow \text{cexec } p.2 \ p.1 \ s'$

6.2 Transitions with continuations

The reduction relation rebuilds a whole command at each step. That is awkward to relate to a machine that just moves a program counter.

Split the configuration in two: the **sub-command under focus**, and a **continuation** recording what remains to be done.

inductive cont **where**

```
| stop
| seq  (c : com) (k : cont)
| while (b : bexp) (c : com) (k : cont)
```

`applyCont k c` rebuilds the whole command by placing `c` leftmost in the nest of sequences described by `k`.

Three kinds of rule

```
inductive step : com × cont × Store → com × cont × Store → Prop where
  -- computation
  | assign : step (.assign x a, k, s) (.skip, k, update x (aeval s a) s)
  | ifthenelse : step (.ifthenelse b 1c 2c, k, s)
    ((if beval s b then 1c else 2c), k, s)
  | while_done : beval s b = false → step (.while b c, k, s) (.skip, k, s)
  -- focusing
  | seq : step (1c ;; 2c, k, s) (1c, .seq 2c k, s)
  | while_true : beval s b = true → step (.while b c, k, s) (c, .while b c k, s)
  -- resumption
  | skip_seq : step (.skip, .seq c k, s) (c, k, s)
  | skip_while : step (.skip, .while b c k, s) (.while b c, k, s)
```

Computation evaluates an expression and acts on the result; *focusing* descends into a sub-command; *resumption* pops the continuation when the focus is skip.

Aside: continuations extend painlessly

Suppose we add C's break. No existing rule changes; we add two resumption rules:

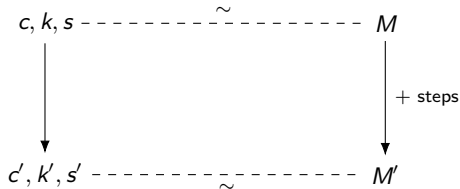
```
| break_seq    : step (.break, .seq c k, s)      (.break, k, s)
| break_while  : step (.break, .while b c k, s) (.skip, k, s)
```

The first says break *floats up* through pending sequences, skipping the computations they contain. When it meets a while continuation it has found its enclosing loop; the second discards that continuation and turns the break into skip.

That is the entire semantics of break. continue is an exercise; labelled break lbl is another.

Simulation diagrams

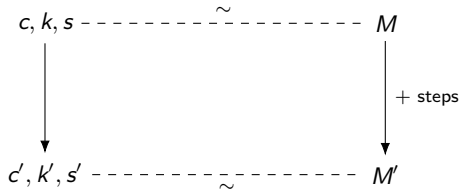
We want: every source step is matched by machine steps, preserving a relation $\sim$ between configurations.



Take the diagram with *one or more* machine steps (+) and divergence transfers immediately: infinitely many source steps give infinitely many machine steps.

Simulation diagrams

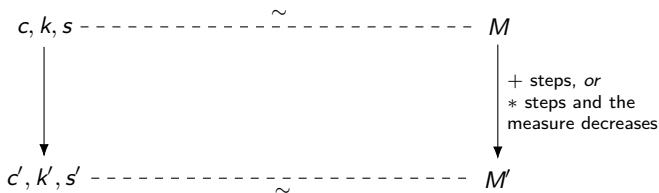
We want: every source step is matched by machine steps, preserving a relation $\sim$ between configurations.



Take the diagram with *one or more* machine steps (+) and divergence transfers immediately: infinitely many source steps give infinitely many machine steps. But our compiler does not satisfy it. Some source steps generate *no* machine transition at all — entering a sequence, or testing a loop that compiles to nothing.

Stuttering, and how to control it

So we allow *zero* machine steps, but only finitely often in a row:



A *measure* on source configurations, strictly decreasing on stuttering steps, rules out an infinite run of them. Without it, a compiler that emitted `halt` for every loop would satisfy the diagram.

Finding a measure is a small black art. Here: the size of the command in focus plus the sizes of the commands recorded in the continuation.

Relating continuations to code

When the machine finishes the code for c , the instructions it meets next should carry out the pending computations in k , then halt.

```
inductive compileCont (C : Code) : cont → Int → Prop where
  | stop : instrAt C pc = some .halt → compileCont C .stop pc
  | seq  : CodeAt C pc (compileCom c) → pc' = pc + codelen (compileCom c) →
    compileCont C k pc' → compileCont C (.seq c k) pc
  | while : instrAt C pc = some (.branch d) → pc' = pc + 1 + d →
    CodeAt C pc' (compileCom (.while b c)) → ... →
    compileCont C (.while b c k) pc
  | branch : instrAt C pc = some (.branch d) → pc' = pc + 1 + d →
    compileCont C k pc' → compileCont C k pc
```

The last rule matters: a continuation may sit behind a *chain of unconditional jumps*.

The matching relation and the measure

```
inductive matchConfig (C : Code) : com × cont × Store → Config → Prop where  
  | intro : CodeAt C pc (compileCom c) →  
           compileCont C k (pc + codelen (compileCom c)) →  
           matchConfig C (c, k, st) (pc, [], st)
```

Same store; empty machine stack; code for c at pc ; code carrying out k after it.

```
def comSize : com → Nat  
  | .skip | .assign _ _ => 1  
  | .seq  $_1 c_2 c$  | .ifthenelse  $_1 c_2 c$  => comSize  $_1 c$  + comSize  $_2 c$  + 1  
  | .while  $_1 c$  => comSize  $_1 c$  + 1
```

```
def measure : com × cont × Store → Nat  
  | (c, k, _) => comSize c + contSize k
```

The simulation theorem

```
theorem simulation_step {C : Code} {1ic 2ic : com × cont × Store}
  {1mc : Config}
  (hstep : step 1ic 2ic) (hmatch : matchConfig C 1ic 1mc) :
  ∃ 2mc, (Plus (transition C) 1mc 2mc
    v (transitions C 1mc 2mc ∧ measure 2ic < measure 1ic))
    ∧ matchConfig C 2ic 2mc
```

Seven cases, one per rule of step.

- assign and skip_while take the + branch.
- seq, ifthenelse, while_done, while_true, skip_seq take the *-with-measure branch.

This is the hard work of the development. Everything below is a corollary.

Corollary 1: termination, again

theorem simulation_steps (h : Star step $_1ic _2ic$) :
 $\forall \{ _1mc \}, \text{matchConfig } C _1ic _1mc \rightarrow$
 $\exists _2mc, \text{transitions } C _1mc _2mc \wedge \text{matchConfig } C _2ic _2mc$

theorem compileProgram_correct_terminating_2 {c : com} {s s' : Store}
 (h : Star step (c, .stop, s) (.skip, .stop, s')) :
 machineTerminates (compileProgram c) s s'

A second proof of the §4 result, now from the continuation semantics.

Corollary 2: divergence

theorem compileProgram_correct_diverging {c : com} {s : Store}
 (h : Infseq step (c, .stop, s)) : machineDiverges (compileProgram c) s

The bridge is a lemma saying that from a diverging source configuration the machine can always make *at least one* transition into a configuration that again matches a diverging source configuration:

theorem simulation_infseq_inv : $\forall (n : \text{Nat}) \{1ic\ 1mc\},$
 Infseq step 1ic $\rightarrow$ matchConfig C 1ic 1mc $\rightarrow$ measure 1ic $< n \rightarrow$
 $\exists\ 2ic\ 2mc, \text{ Infseq step } 2ic \wedge \text{ Plus (transition C) } 1mc\ 2mc$
 $\wedge \text{ matchConfig C } 2ic\ 2mc$

The bound n on the measure turns this into an ordinary induction: consecutive stuttering steps must run out.

Interlude: divergence in Lean

Coq defines divergence with `CoInductive` — a *greatest* fixpoint. Lean 4 has no coinductive types, so we take the greatest fixpoint by hand. For $\Phi(X) = \{a \mid \exists b, R a b \wedge b \in X\}$, Knaster–Tarski gives

$$\nu\Phi = \bigcup \{X \mid X \subseteq \Phi(X)\}.$$

Written out, that is the definition:

```
def Infseq (R : A → A → Prop) (a : A) : Prop :=  
  ∃ X : A → Prop, X a ∧ ∀ x, X x → ∃ y, R x y ∧ X y
```

X is an *invariant*: from every member you can step and stay inside. So you can never get stuck.

An inductive definition would be the *least* fixpoint — which, having no base case, is empty.

The coinduction principle is the definition

theorem Infseq.coinduction {X : A → Prop}
 (hX : ∀ x, X x → ∃ y, R x y ∧ X y) (ha : X a) : Infseq R a :=
 ⟨X, ha, ⟩hX

In Coq this needs `cofix` and a guardedness check. Here it is just repackaging.

The constructor and destructor are recovered as lemmas, so $\text{Infseq} = \Phi(\text{Infseq})$ really is a fixpoint.

And the payoff, in the divergence proof: the invariant supplied is

“this machine configuration matches some diverging source configuration”

— that is, the invariant *is* the simulation relation, named and visible.

8.1 Smart constructors

Build the simplified expression instead of building then simplifying:

```
def mkPlusConst (a : aexp) (n : Int) : aexp :=  
  if n = 0 then a else  
    match a with  
    | .const m => .const (m + n)  
    | .plus a (.const m) => .plus a (.const (m + n))  
    | _ => .plus a (.const n)
```

mkPlus uses associativity and commutativity to expose the pattern “expression plus constant”;
mkMinus normalises “expression minus constant” into “expression plus the opposite constant”.

```
theorem mkPlus_sound (s : Store) (1a 2a : aexp) :  
  aeval s (mkPlus 1a 2a) = aeval s 1a + aeval s 2a
```

Commands benefit too: mkIfthenelse and mkWhile take known conditions into account.

8.2 Abstract stores

An *abstract store* records what the analysis knows. If it maps x to n , then x certainly holds n at run time; if it maps x to nothing, x may hold anything.

So *less information* means *larger*, and the top element is the empty map.

```
def Matches (s : Store) (S : AStore) : Prop :=  
   $\forall x\ n, S.find\ x = \text{some } n \rightarrow s\ x = n$ 
```

```
def Le ( $_1S\ _2S$  : AStore) : Prop :=  
   $\forall x\ n, _2S.find\ x = \text{some } n \rightarrow _1S.find\ x = \text{some } n$ 
```

Matches is the concretisation relation — a bit like “a term has a type”. The join keeps only the facts two stores agree on.

(Coq uses FMaps; Lean’s core has no finite maps, so these are association lists.)

Abstract evaluation

```
def Aeval (S : AStore) : aexp → Option Int
| .const n => some n
| .var x   => S.find x
| .plus 1a 2a => match Aeval S 1a, Aeval S 2a with
| some 1n, some 2n => some (1n + 2n)
| _, _ => none
| ...
```

some v means “the value is certainly v ”; none means “unknown”. Soundness relates the two worlds:

```
theorem Aeval_sound (hm : Matches s S) :
  ∀ (a : aexp) {n : Int}, Aeval S a = some n → aeval s a = n
```

The analysis

```
def Cexec (S : AStore) (c : com) : AStore :=  
  match c with  
  | .skip => S  
  | .assign x a => AStore.update x (Aeval S a) S  
  | .seq 1c 2c => Cexec (Cexec S 1c) 2c  
  | .ifthenelse b 1c 2c => match Beval S b with  
    | some true  => Cexec S 1c  
    | some false => Cexec S 2c  
    | none       => AStore.join (Cexec S 1c) (Cexec S 2c)  
  | .while _ 1c => fixpoint (fun X => AStore.join S (Cexec X 1c)) S
```

Loops need a fixpoint — executing the loop in the abstract until the abstract state stops changing. For now, the engineer's answer: iterate twenty times, and fall back on top (know nothing). Imprecise, but sound, and §9 will do better.

Soundness of the analysis

theorem fixpoint_sound (F : AStore $\rightarrow$ AStore) (initS : AStore) :
Le (F (fixpoint F initS)) (fixpoint F initS)

theorem Cexec_sound : $\forall$ (c : com) { $s_1 s_2$: Store} { S_1 : AStore},
cexec s_1 c s_2 $\rightarrow$ Matches s_1 S_1 $\rightarrow$ Matches s_2 (Cexec s_1 c)

The *only* property of fixpoint used is that it returns a post-fixpoint. Precision is not needed for soundness.

The while case needs an unusual induction: structural on the command, plus an inner induction on the iterations actually performed. That pattern mirrors the structure of Cexec itself.

8.3 The optimisation

```
def cpCom (S : AStore) (c : com) : com :=  
  match c with  
  | .skip => .skip  
  | .assign x a => .assign x (cpAexp S a)  
  | .seq 1c 2c => .seq (cpCom S 1c) (cpCom (Cexec S 1c) 2c)  
  | .ifthenelse b 1c 2c =>  
    mkIfthenelse (cpBexp S b) (cpCom S 1c) (cpCom S 2c)  
  | .while b 1c =>  
    let sfix := Cexec S (.while b 1c)  
    mkWhile (cpBexp sfix b) (cpCom sfix 1c)
```

S is what is known *before* c , so it must be threaded exactly as the analysis threads it.

```
theorem cpCom_correct_terminating :  $\forall (c : \text{com}) \{1s \ 2s \ 1S\},$   
  cexec 1s c 2s  $\rightarrow$  Matches 1s 1S  $\rightarrow$  cexec 1s (cpCom 1S c) 2s
```

9.1 From Knaster–Tarski to computation

Let F be monotone on a set with a decidable equality, a transitive order, and a **well-founded strict order** — every strictly ascending chain is finite.

Knaster–Tarski. F has a fixpoint.

The proof is already an algorithm: start from a pre-fixpoint $x \leq F(x)$; either $x = F(x)$ and we are done, or $F(x)$ is strictly above x and is itself a pre-fixpoint, so recurse. Well-foundedness makes the recursion terminate.

9.1 From Knaster–Tarski to computation

Let F be monotone on a set with a decidable equality, a transitive order, and a **well-founded strict order** — every strictly ascending chain is finite.

Knaster–Tarski. F has a fixpoint.

The proof is already an algorithm: start from a pre-fixpoint $x \leq F(x)$; either $x = F(x)$ and we are done, or $F(x)$ is strictly above x and is itself a pre-fixpoint, so recurse. Well-foundedness makes the recursion terminate.

But an existence proof $\exists x, x = F(x)$ gives no way to *compute* x . Pattern-matching on it to extract the witness is not allowed.

Producing the value, not just its existence

Return a *subtype* instead — the value packaged with its specification:

```
def iterate (Fmon :  $\forall x y, L.le\ x\ y \rightarrow L.le\ (F\ x)\ (F\ y)$ ) :  
   $\forall (x : A), L.le\ x\ (F\ x) \rightarrow \forall (z, L.le\ (F\ z)\ z \rightarrow L.le\ x\ z) \rightarrow$   
    {  $y : A // L.eq\ y\ (F\ y) \wedge \forall z, L.le\ (F\ z)\ z \rightarrow L.le\ y\ z$  } :=  
L.gt_wf.fix fun x ih hpre hsmall =>  
  match hb : L.beq x (F x) with  
  | true  => {x, L.beq_eq hb, }hsmall  
  | false => ih (F x) (hpre, L.beq_neq hb (Fmon _ _ hpre))  
    (fun z hz => L.le_trans (Fmon _ _ (hsmall z hz)) hz)
```

It carries a pre-fixpoint *and* a proof that it is below every post-fixpoint, and returns a fixpoint that is still below every post-fixpoint — the *least* one.

Coq writes this with Program Fixpoint, leaving holes for the proofs. Lean uses WellFounded.fix directly.

9.2 Application: the hard part

To use this for abstract stores we must show the strict order is well founded. Measure a store by the number of *distinct* variables it pins down:

theorem Le_card {S T : AStore} (h : Le T S) :
card S ≤ card T ∧ (card S = card T → AEq T S)

theorem Gt_wf : WellFounded (**fun** S S' => Le S' S ∧ ¬ AEq S' S)

Going strictly up in the order means losing at least one fact, so the measure strictly decreases — and Nat is well founded.

Coq reasons about the *cardinal* of a finite map. With association lists we reason about the length of the deduplicated key list, which needs two combinatorial lemmas about duplicate-free lists.

9.2 The second obstacle: no bottom

Abstract stores have no smallest element, so we cannot iterate from one.

But for the functions we care about there *is* a natural starting point: the store *init* describing what is known on entry to the loop. Iterate $X \mapsto \text{init} \sqcup F(X)$, which is above *init* by construction.

```
def fixpointJoin (init : AStore) (F : AStore → AStore) (hF : Increasing F)
  : AStore
```

```
theorem fixpointJoin_sound :
  Le init (fixpointJoin init F hF)
  ∧ Le (F (fixpointJoin init F hF)) (fixpointJoin init F hF)
```

```
theorem fixpointJoin_smallest (S : AStore) :
  Le (AStore.join init (F S)) S → Le (fixpointJoin init F hF) S
```

Definition and proof, simultaneously

Now try to rebuild the analyser using `fixpointJoin...` and it does not typecheck: *only increasing functions have fixpoints*, and the loop case needs to know the body is increasing.

So the function must return its own proof:

```
def CexecM : com → { F : AStore → AStore // Increasing F }
| .skip => (id, fun _ _ h => )h
| .assign x a => (fun S => AStore.update x (Aeval S a) S,
                 fun _ _ h => update_increasing x a )h
| .seq c1 c2 => (fun S => (CexecM c2).1 ((CexecM c1).1 S), )...
| .while _ c => (fun S => fixpointJoin S (CexecM c).1 (CexecM c).2,
                 fun _ _ h => fixpointJoin_increasing _ _ )h
```

Definition and monotonicity proof are constructed in one go — there is no way to separate them.

10.1 Liveness analysis

A variable is **live** at a program point if some later instruction may read it before it is overwritten. An assignment to a variable that is not live is *dead*.

This is a **backward** analysis — contrast constant propagation: live c L is the set live *before* c , given that L is live *after* it.

```
def live (c : com) (L : IdentSet) : IdentSet :=  
  match c with  
  | .skip => L  
  | .assign x a => if x ∈ L then remove x L ++ fvAexp a else L  
  | .seq  $_1c$   $_2c$  => live  $_1c$  (live  $_2c$  L)  
  | .ifthenelse b  $_1c$   $_2c$  => fvBexp b ++ (live  $_1c$  L ++ live  $_2c$  L)  
  | .while b c => fixpoint (fun x => (fvBexp b ++ L) ++ live c x)  
    (fvCom (.while b c) ++ L)
```

Note the assignment case: if x is live after, it stops being live before, and everything a reads becomes live.

10.2 The optimisation

```
def dce (c : com) (L : IdentSet) : com :=  
  match c with  
  | .skip => .skip  
  | .assign x a => if x ∈ L then .assign x a else .skip  
  | .seq 1c 2c => .seq (dce 1c (live 2c L)) (dce 2c L)  
  | .ifthenelse b 1c 2c => .ifthenelse b (dce 1c L) (dce 2c L)  
  | .while b c => .while b (dce c (live (.while b c) L))
```

On Euclidean division, keeping only "r" live at the end:

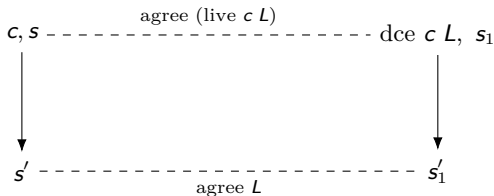
r := a;	==>	r := a;
q := 0;		skip;
while b <= r do		while b <= r do
r := r - b;		r := r - b;
q := q + 1		skip

Keeping "q" live instead, nothing can be removed.

10.3 Correctness

The optimised program runs in a store that may *differ* from the original on dead variables — that is the whole point. So the invariant is agreement on the live ones.

def agree (L : IdentSet) (s₁ s₂ : Store) : Prop :=
 $\forall x, x \in L \rightarrow s_1 x = s_2 x$



theorem dce_correct_terminating {s c s'} (h : cexec s c s') :
 $\forall (L : \text{IdentSet}) (s_1 : \text{Store}), \text{agree } (\text{live } c \ L) \ s \ s_1 \rightarrow$
 $\exists s'_1, \text{cexec } s_1 (dce \ c \ L) \ s'_1 \wedge \text{agree } L \ s' \ s'_1$

What we proved

- A compiler from IMP to a stack machine, with semantic preservation for *terminating and diverging* programs.
- Constant propagation: a forward dataflow analysis, its soundness, and the optimisation built on it.
- Dead code elimination: a backward analysis and its optimisation.
- The fixpoint theory that makes the analyses precise, with a terminating algorithm.

All machine-checked, with no sorry, in about 3 000 lines of Lean 4 and no dependencies beyond the core library.

What scales, and what does not

The *methods* scale: this is how CompCert — a verified C compiler, with a dozen intermediate languages and many more optimisations — is built.

What gets harder:

- real languages have memory, pointers, function calls, undefined behaviour;
- real machines have registers and a finite number of them;
- simulation diagrams compose, but the invariants get big.

What stays the same: semantics first, then a simulation relation, then a measure to control stuttering.

Further reading

- Xavier Leroy. *Formal verification of a realistic compiler*. Comm. ACM 52(7):107–115, 2009.
- Xavier Leroy. *A formally verified compiler back-end*.
- Benjamin Pierce et al. *Software Foundations*.
- Yves Bertot. *Structural abstract interpretation, a formal study using Coq*.

Original course and Coq sources: <https://xavierleroy.org/courses/EUTypes-2019/>